Chapter 11. Conclusions and the Future

At this point you might be expecting (and be glad) that this is the end of the book. Not quite, because during my time writing this book I collected a smorgasbord of tidbits and ideas for future work. In the first half, I delve into some tips and tricks that I have accumulated that didn't fit into other parts of the book. In the second half I outline the current challenges and provide direction for future research.

Tips and Tricks

I've said this a few times now: RL is hard in real life because of the underlying dependencies on machine learning and software engineering. But the sequential stochasticity adds another dimension by delaying and hiding issues.

Framing the Problem

Often you will have a new industrial problem where the components of the Markov decision process are fuzzy at best. Your goal should be to refine the concept and prove (at least conceptually) the viability using RL for this problem.

To start, try visualizing a random policy acting upon a notional environment. Where does it go? What does it interact with? When does it do the right thing? How quickly does it find the right thing to do? If your random policy never does the right thing, then RL is unlikely to help.

After trying a random policy, try to solve the problem yourself. Imagine *you* are the agent. Could you make the right decisions after exploring every state? Can you use the observations to guide your decisions? What would make your job easier or harder? What features could you generate from these observations? Again, if you can't notionally come up with vi-

able strategies, then RL (or machine learning, for that matter) is unlikely to work.

Use baselines to frame performance. Start with a random agent. Then use a cross-entropy method (which is a simple algorithm that recalls the best previous rollout from random exploration). Then try a simple Q-learning or policy gradient algorithm. Each time you implement a new algorithm, treat it like a scientific experiment: have a theory, test that theory, learn from the experiment, and use it to guide future implementations.

Simplify the task as much as humanly possible, then solve that first. Go for the quick wins. By *simplify*, I mean reducing the state and action space as much as practicable. You can hand engineer features for this or use your domain knowledge to chop out parts of the state that don't make sense. And simplify the goal or the problem definition. For example, rather than trying to pick up an object with a robot, try training it to move to a position first. And simplify the rewards, too. Always try to remove the sparsity by using metrics like the distance to the goal, rather than a positive reward when the agent reaches the goal. Try to make the reward smooth. Remove any outliers or discontinuities; these will make it harder for your model to approximate.

Ask yourself whether you need to remember previous actions to make new ones. For example, if you need a key to open a door, then you need to remember that you have picked up a key. If you want to place an item in a box, you need to pick it up first. These situations have states that must be visited in a specific order. To provide this capability you should incorporate a working memory, and one common implementation is via recurrent neural networks.

Don't confuse this with two actions, though. It's easy to fall into the trap of thinking that you need memory, but in fact you have actions to reach these states directly. For example, you would imagine that before moving forward in a car you need to turn the engine on. This sounds like ordered tasks again, but it is not, because you probably have separate actions to turn the car on and move forward. An agent without memory is fully capable of learning to keep the car turned on in order to move forward. This highlights a way of removing the need for memory, too. If you can find an action that provides a shortcut to this state, then you might be able to do away with the working memory.

Also remember hierarchical policies at this point. Consider if your application has disparate “skills” that could be learned to solve the problem. Consider solving them independently if you can, which is conceptually and practically simpler than using hierarchical RL and the end result may the same.

Your Data

I think the vast majority of development time is spent retraining models after tweaking some hyperparameter or altering features, for example. For this reason, and this almost goes without saying, you should try to do things right the first time. This only comes with experience: experience with RL in general, in your domain, and with your problem. But here are a few tips that should provide a bit of a shortcut.

Like in machine learning, scale and correlation is really important. Not all algorithms *need* rescaling (trees, for example), but most do, and most expect normally distributed data. So in general you should always rescale your observations to have a mean of zero and a standard deviation of one. You can do this with something as simple as a rolling mean and standard deviation, or if your input is stationary or bounded, you can use those observed limits. Be aware that the statistics of your data may change over time and make your problem nonstationary.

Also try to decorrelate and clean your observations. Models don’t like correlation in time or across features, because they represent complex biases and duplicate information. You can use the z-transform, principal component analysis, or even a simple diff between observations to decorrelate single features. You use your data analysis skills to find and remove correlations between observations.

Monitor all aspects of your data. Look at the reward distributions; make sure there are no outliers. Look at the input data, again, plot distributions or visualizations, and make sure the input data isn’t wild. Look at the predictions being made by various parts of the model. Is the value function predicting well? What about the gradients? Are they too big? Look at the entropy of the action space: is it exploring well, or is it stuck on certain actions? Look at the Kullback–Liebler divergence: is it too big or too small?

Think about increasing increasing the amount of discretization, in other words, increase the time between actions. This can make training more stable, because it's less affected by random noise, and speed it up, because there are fewer actions to compute. The de facto frame skipping in the Atari environments is an example of this. A good rule of thumb is to imagine yourself picking actions; at what rate do you need to make decisions to produce a viable policy?

Training

Training can be quite a demoralizing process. You think you have a good idea only to find that it doesn't make any difference and it took you a day to implement and retrain. But at least this is visible. One of the biggest issues in RL right now is that it is easy to overfit the development of an algorithm to your specific environment and therefore produce a policy that doesn't generalize well.

Try to develop your algorithm against a range of environments. It is very easy to overfit your development to the environment you use for ad hoc testing. This takes discipline, but try to maintain a set of representative environments, from simple to realistic, that also have different setups, like larger or smaller, easier or harder, or fully observable or partially observable. If your algorithm works well across all of them, then that gives you confidence that it is able to generalize.

Use different seeds. I've sometimes thought I was doing well in an environment, but it turned out that one specific seed was massively overfitting and happened to stumble across a viable policy purely by chance. When you use different seeds, this forces your agent to explore in a slightly different way than it did before. Of course, your algorithm should perform similarly irrespective of the seed value, but don't be surprised if it doesn't.

Test the sensitivity of your hyperparameters. Again, you should be aiming for an algorithm that is robust and degrades gracefully. If it works only with a very specific set of hyperparameters then this is an indication of overfitting again.

Play with the learning rate schedules. A fixed value rarely works well because it encourages the policy to change, even when it is doing well. This

can have the effect of knocking the agent away from an optimal policy.

Evaluation

Evaluation tends to be one of the least problematic phases because you should already have a good idea of how well things are going from all the monitoring you did during training. But there is still enough fuel left to get burnt.

Be very careful drawing conclusions when comparing algorithms. You can use statistics to quantify improvement (see "Statistical policy comparisons") but I find that I make fewer mistakes when visualizing performance. The basic idea is that if you can see a significant performance increase, where there is clear space between the performance curves over many different seeds, then it's quite likely to be better. If you don't see a gap, then run more trials with more seeds or simply don't trust it. This sounds simple enough but in really complex environments it is not. Often, through no fault of your algorithm, you can end up with a really bad run. This can happen in a complex environment where a bad initial exploration led to states that are really hard to get out of. Take a look at the states that your agent is choosing to explore; you might want to alter the exploration strategy.

No one measure or plot can truly prove performance. I find that a holistic combination of experience with that domain and environment and intuition about how an algorithm *should* behave dictates whether I believe there to be a change in performance. This is why it is important to have baselines to compare yourself against, because it provides another piece of evidence.

Remember that some policies are stochastic by default—many policy gradient algorithms, for example—so evaluate using deterministic versions of these policies if you want a pure performance measure. Similarly, if you have any random exploration in your policy—ϵ-greedy, for example—make sure you disable it. Even though you might disable it during evaluation, consider retaining the stochasticity in the real environment if there is a chance that the optimal policy could change over time.

Deployment

I think the key to quickly developing successful applications is defined by the ability to combine off-the-shelf components and modules to solve a particular problem. This necessitates a certain amount of composability, which is primarily driven at the software level through good coding practices. The idea is that you look at your problem, pick the components that would help—intrinsic motivation or a replay buffer for example—stick them all together through nicely encapsulated interfaces, and deploy them in unison. At the moment, unfortunately, this dream is a long way away. Any production deployment will involve significant software engineering and DevOps expertise.

Frameworks help here, too, because many of them have the desire to be composable. But that usually means that they are composable within the bounds of their codebase. Frameworks often struggle to interact with external components because of a lack of standards. OpenAI's `Gym` is an obvious exception to this.

So think hard about your architecture, because I think this is the key to successful deployment and operation of a real-life product.

Debugging

Despite RL algorithms fueling much of the publicity surrounding artificial intelligence, deep RL methods are "brittle, hard to reproduce, unreliable across runs, and sometimes outperformed by simple baselines."[1] The main reason for this is that the constituent parts of an algorithm are not well understood: how do the various parts of an implementation impact behavior and performance?

Implementation details are incredibly important. Engstrom et al. find that seemingly small modifications like code-level optimizations can explain the bulk of performance in many popular algorithms. For example, despite the claim that PPO is performant due to update clipping or constraints, these researchers proved that the code-level optimizations improved performance *more* than the choice of algorithm (they compared TRPO to PPO).[2]

This leads to the obvious conclusion that bugs impact performance. For example, during OpenAI's random network distillation research (see "Random embeddings (random distillation networks)"), Cobbe et al. noticed that seemingly small details made the difference between an agent that gets stuck and one that can solve a difficult Atari game. They noticed one issue where the value function approximation, which should be stable after enough training, was oscillating. It turned out that they were accidentally zeroing an array that was used to produce the curiosity bonus, which meant that the value function was pulled lower at the start of each episode. This explained the oscillation and, once fixed, dramatically improved performance due to the stable value estimates.[3]

So the question is, how do you prevent these bugs? Well, every single line of code introduces a possibility of a bug, so the only way to eliminate all bugs is to have no code at all. I preach this mantra a lot during my productization projects with Winder Research, which also has the added benefit of reducing the operational burden. Obviously you need code, but the point I'm making is that less is better. If you have an option of using an off-the-shelf library or component, then use it, even if it doesn't quite meet all your requirements. Similarly, if you are writing code or developing a framework, prefer less code over fancy design. This is why I love languages like Go or Python. They actively promote simplicity because they don't have all the so-called sophistication of a language like C++ or Java.

Beyond this, the normal debugging practices of tracing, visualizing your data, testing, and `print` statements will become your day-to-day experience of RL. At its heart, debugging is just a manifestation of the scientific method: you observe something strange (visualizations are really important), you develop a theory to explain the behavior, you develop ways to test that theory (`print` statements), and you evaluate the results. If you want to learn more, pick up a book on debugging software; some recommendations are in "Further Reading".

RL is sequential, which means that bugs might not manifest for some time, and RL is stochastic, which means that bugs can be hidden by the noise of the learning process. Combine this with the underlying dependencies of machine learning and software engineering and you end up with a volatile witch's cauldron of bubbling bugs.

If you were to speak to your project manager and estimate how long you would spend developing an algorithm for a new problem, you would estimate something like the top part of Figure 11-1; most of the time is spent on research and implementation, right? No. The vast majority of time is spent debugging, attempting to figure out why your agent can't solve the simplest thing. The bottom part of Figure 11-1 shows something more representative, with the added time that is required to reimplement your code properly, with clean code and tests. The measurements I picked are arbitrary, but they are representative, in my experience.

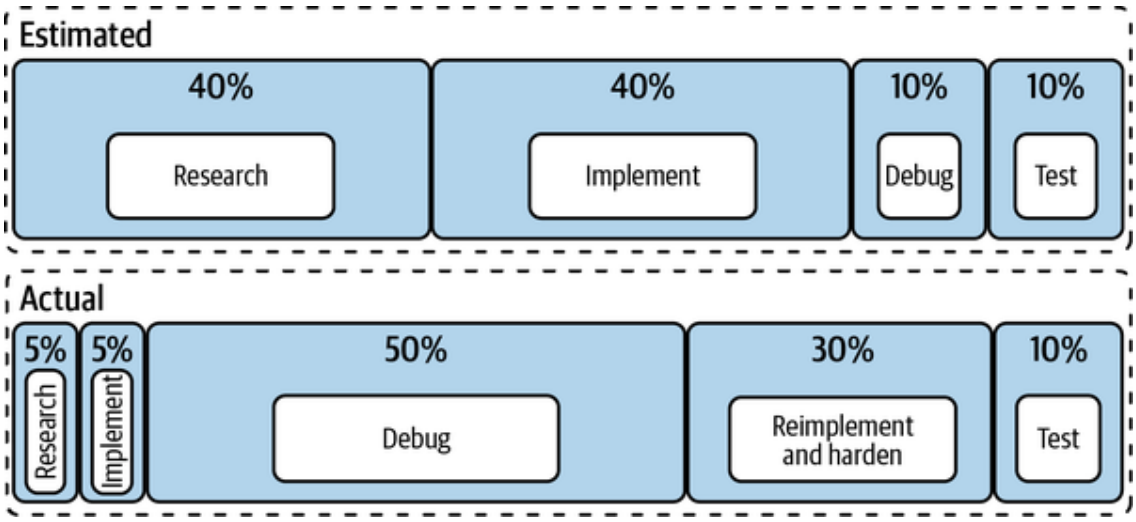

Figure 11-1. A comparison of expected versus actual time spent on activities during algorithm development. Inspired by Matthew Rahtz.[1]

${ALGORITHM_NAME} Can't Solve ${ENVIRONMENT}!

Even though these algorithms can achieve amazing feats, with examples that you might even consider intelligent, it is important to remember that they are not magic. They cannot solve every problem and they certainly don't do it automatically.

If you are using RL in industry, you must take a holistic view. It is very likely that you can do something to dramatically simplify the problem. The main reason why RL methods fail is not because the algorithm isn't good enough, but because the problem isn't suited to being solved in this way. For example, the `MountainCar-v0` environment is notoriously difficult for RL algorithms to solve, despite being conceptually simple. This is not a problem with the algorithm; the value function is surprisingly simple. This is an exploration and a credit assignment problem. If you developed a better way to explore or changed the reward to encourage move-

ment up the hill, any of these algorithms can solve the problem in no time.

There is no single solution to solving environments either. Sometimes the problem lies with the observation space, sometimes the exploration, sometimes a hyper-parameter. Welcome to RL, where intelligent agents automate complex strategic goals, sometimes. My only recommendation is to use this book to guide you, one experiment at a time, toward a tractable problem. If you forced me to pick one, I'd start with a simplification of the problem or observation space. Get it simple enough so you can visualize what is going on and then solve that simple problem. With that in your bag, you'll probably have enough domain knowledge to know the issues with the more complex version.

Monitoring for Debugging

Monitoring in production, for operational purposes, does not provide enough low-level information to help you debug an RL algorithm. When you are developing your agent you will want to log, plot, and visualize as much as you can. Following is a list of tips to help you when you need more information:

- Log and visualize observations, actions, and rewards. Actions should have high entropy to encourage exploration. The rewards for states should not be the same; if they are the agent can't tell which states are more valuable. Check the observation presented to the agent; it's easy to accidentally throw away important information. For example, check that a grayscale transformation of an image preserves features within that image; green and red traffic lights are clearly different in color, but not so much in grayscale.
- Evaluate the value function for different parts of your state space. If you can, visualize the whole space. If not, pick some representative states and plot them. Is it overfitting?
- Plot the value function over time (for a reasonable number of states). How does it change? Does it get stuck? Does it oscillate? What is the variance during a run? What is the variance over several runs? The estimate should alter over time and then stabilize. Remember that when the policy changes, the perfect value function changes, too. The

value function approximation should remain stable enough to cope with the nonstationarity, but flexible enough to react to changes.
- Poor or buggy neural network implementations make RL look bad. Start with something simple, a linear approximator or equivalent single-layer neural network. Make sure everything works as expected before trying more complex networks. Look at a good practical deep learning book for help with this (see "Further Reading").
- Make sure everything works with fixed seeds and single threading, before you scale up to multiple seeds or multithreading/multinode runs.
- Start with hyperparameters that are known to work well from your past experience with other implementations.

The Future of Reinforcement Learning

What is the current opinion of RL right now? This makes me think of Andrew Ng's almost seminal talk on practical deep learning, where he manages to perfectly balance a healthy dose of short-term cynicism with long-term optimism.[5] This left me appreciating the complexities of the subject, especially when it comes to industrial implementations, but at the same time the optimism made these issues feel like it was just part of the process. The challenges are there, so accept them, deal with them in the best way you can, and move on. Sometimes deep learning doesn't make sense, and that's OK. But when it does, the confluence of model and data are a beauty to behold.

I think RL is in this position right now. We're on the cusp of where industrial productization and operationalization are possible, but tricky. There are still many cases where simpler algorithms outperform RL; for example, if you are able to play your environment offline, then you could just use a Monte Carlo tree search (MCTS) algorithm, like AlphaGo, to evaluate your possible moves. For example, an experiment in 2014 smashed the performance of DQN using MCTS and it took some time before RL methods could catch up.[6] But this is OK. RL isn't a silver bullet, just like machine learning hasn't had babies and created artificial intelligence yet. It's just a tool. And like a carpenter, you should use the best tool for your job.

RL Market Opportunities

Deep learning has become mainstream, so even my enterprise clients are investing in operationalization. I believe that RL will follow within the next few years. This opens up new markets and opportunities. Unfortunately, like machine learning, RL is a cross-industry technique, not a solution. You will need domain expertise, industry-specific market awareness, and a dollop of inspiration to exploit RL.

There are already signs that RL is especially well suited to specific domains, however. Robotics is one obvious one. But automated bidding and recommendations are others that could overtake robotics in terms of global revenue. There hasn't been any industry-wide research specific to RL yet, but that will come if RL is successful.

The market research firm Gartner reckons that RL is still on the rise of the hype curve and that a third of enterprises are looking to automate strategic decision making (which they call decision intelligence) by 2023.[7,8] They are suitably vague about market values, especially when it comes to quantifying the impact of specific technologies, but the worldwide value to business is reported to be many trillions of dollars per year.[9] Interestingly, in some of Gartner's visualizations, they are beginning to separate "agents" and "decision automation" from other forms of more traditional machine learning, which they call "decision support/augmentation," so they can clearly see the emerging value of optimizing sequential decisions.

This promise means that there is value and opportunity in the operational space. Companies won't be able to use the current crop of machine-learning-as-a-service offerings because most are not well suited toward continuous learning and sequential decision making. Some will adapt, of course, but there are opportunities for companies to fill this space.

Open source projects won't be far behind, either, and could even be in front in many cases. There is an argument to be made that only open source projects can be truly successful because, like machine learning, global adoption needs open collaboration. Otherwise your needs and challenges might conflict with proprietary processes and opinions. If you are considering this, make sure you position your product some distance away from what large vendors might be interested in. And make sure you

reflect on how you and the global community can mutually gain from open source development.

This also means that companies will need RL support and expertise for a long time into the future. It could be as much as 10 years before RL is a commodity. So consultants and engineers with demonstrable RL expertise, like those from my company, will be sought after. If you are thinking about specializing in RL as a career, this would not be a bad decision, I don't think, because of the obvious opportunities and projections just discussed. But don't let this lull you into a false sense of security. RL, ML, and software engineering is hard, especially when you try to move these things into production. And like in any knowledge-based industry, the goal posts are constantly moving. You have to accept this and move with them to assure a long-term position in this highly competitive field. To put in this much effort you really have to love it. You have to enjoy the craft and the challenge of adversity, the challenge of being met with a specific industrial situation that nobody, ever, has dealt with before.

Future RL and Research Directions

RL has many challenges, as I have discussed throughout the book. Don't let these prevent you from exploring the potential of RL, because I truly believe that the next level of artificial intelligence is going to be driven by these ideas; they are just waiting for breakthrough applications. But hopefully I have clearly marked which parts of RL can slow you down or cause you to stumble when working on an industrial problem. These problems mostly translate into future research opportunities.

The research directions can be broadly split in two: industrial and academic research. Industrial research is comprised of ways to improve the development and operational efficiency and is driven by business needs. Academic research focuses more on improving or redefining fundamental understanding and is driven by a theoretical notion of novelty or advancement, as defined by their peers. This means that academic researchers are not going to invest their time in improving their code's readability, for example. Likewise, industrial researchers are unlikely to stumble across anything truly groundbreaking. But of course, there are people that sit in between.

WARNING

I believe that there is a problem in the academic machine learning community at the moment. The primary venues for academic success are conferences and journals. The most popular can afford to pick and choose, so reviewers have a lot of power to direct current and future content. And unfortunately, applications are deemed to be of lesser value than, for example, a novel algorithmic improvement. For example, Hannah Kerner has recently spoken about her experience trying to push application-focused papers to conferences, only to find that "the significance seems limited for the machine-learning community."[10] Of course, fundamental improvement is the primary goal for academia and scientific discovery in general, but I feel like the scale has tipped too far.

Research in industry

Starting from an industrial perspective, these are the areas in which I think there are research opportunities and momentum:

Operationalization and architecture

RL is being used more and soon there will be a glut of applications moving into production. I don't think that any of the current machine learning frameworks have the capability and the RL frameworks tend to focus on development, not production. Of course, there are a few exceptions, but there is a lot of room for improvement, particularly on the architecture side.

RL design and implementation

There is already a lot of work in this area—see all the RL frameworks for an example—but many of them tend to focus on the wrong problem. Many of them are attempting to implement RL algorithms, but this is not the goal for an industrial problem. The goal is to find, refine, and build an RL solution to a business challenge. This means that things like feature engineering, reward shaping, and everything else listed in the previous two chapters are also important, possibly more so, than the algorithm. The frameworks need to shift their focus away from algorithm development and toward application development.

Environment curation and simulation

Simulators are obviously crucial for initial development. Accurate simulations provide the ability to transfer knowledge to the real world. As more complex industrial problems are attempted, these will benefit from more realistic simulations. This is already the case for 3D simulation engines, which are capable of simulating scarily good representations of the real world. But for all the other domains, decent, realistic simulators are hard to come by. One reason for this is that they contain a lot of proprietary information. The models that drive the simulations are often built from data that the company owns, so they know that this represents a competitive advantage. One way research could help here is to improve the curation and development process of an environment. This side-steps the data issue but will still help this crucial phase of development.

Monitoring and explainability

The tooling and support surrounding monitoring RL agents is lacking and can be attributed to the lack of operationalization. When in production you need to be certain that your policy is robust and the only way to be sure of that is through adequate oversight. Some applications may even demand explainability, which is quite tricky at the moment, especially in production settings. But more than that, you need tools to help debug the development of applications, tools that give you a view of what is happening inside the agent. The problem with this is that it tends to be algorithm and domain specific, but it is still a worthwhile challenge.

Safety and robustness

This is a hot topic in academic research, too, but the impact of the research is only felt when it is applied to a real-life application. Proving the robustness of your model is important for every application, so I envisage libraries and tools to help solve that problem. Safety is a much bigger concern. I'm less worried about the technical aspects, because I'm confident that engineers can create solutions that are safe by design. I'm concerned about the legal and regulatory impacts, because at this point it becomes more of an ethical problem. Who is liable? How safe does it need to be? How do you test how safe it is?

Applications

And of course, businesses are paying for the application of RL. The research directions discussed here help in delivering that application. There will be a lot of interest and therefore research in applying RL to business challenges. The hard part, like with any technology, is mapping the technology to a business challenge. This is a lot harder than it sounds, because RL is so general; there are many possible applications. So use the suggestions presented in "Problem Definition: What Is an RL Project?" to find a viable RL problem and then prioritize by business value.

Research in academia

From an academic perspective, RL is really exciting, because it feels like we're only a few steps away from something truly revolutionary: an agent that can learn any task as well as a human. This moment is like a bulkhead; once it is breached, this opens up a whole world of possibilities, mostly good, some scary.

Current applications of machine learning and, to a lesser extent, RL, are limited by their inability to utilize the scientific method and retain that knowledge in an intelligent way. If an agent was capable of developing its own theories, testing its own beliefs, and can apply that knowledge to discover new theories, then I think that is the moment where we can generalize RL to solve any sequential problem on any data. And I feel like we're almost there. But to be specific, the challenges that are preventing this goal, from an application perspective, are:

Offline learning

This is probably the biggest challenge. The primary reason why we need simulators is because the agent must test an action to learn a policy. If we can move away from this regime, training a policy without that interaction, this means we can remove the simulator entirely. Just capture log data and train upon that, just like in machine learning. Researchers have already begun to improve this process; look back at Chapter 8, for example. But there is a lot left to be desired. Intuitively, I don't think it is unreasonable to imagine an algorithm that is capable of understanding the stochasticity of an environment, observing examples of an optimal trajectory, then applying a model to fit a generalized trajectory to the data. This

should provide a compromise between finding an optimal policy and retaining enough uncertainty about the states not observed from the log.

Sample efficiency

Improving the sample efficiency of algorithms is desirable because

don't want to get bags of dog and cat food delivered when I don't have any pets. The faster it can learn this preference, the less upset I'm going to be. Again, there are a range of research directions here, from improving exploration to consolidating knowledge, like ensembles, but these are often driven by the domain. It would be great if there was some universal, theoretical efficiency guarantee that algorithms could work toward.

High-dimensional efficiency

Large state and action spaces are still problematic. The large state spaces are awkward because of the amount of computational energy required to reduce it to something useful. I imagine that for certain domains pretrained transformers or embeddings could be used to simplify the state space without having to train the reduction. A large action space is more problematic because agents have to actively sample all available actions at all times to ensure they have an optimal policy. This is easier when actions are related. For example, a single continuous action is infinite, but it is quite easy to solve because you can model the action with a distribution. But many distinct actions cause more of a problem, mainly because this yields a more difficult exploration problem.

Reward function sensitivity

Supervised machine learning is easy because you know what the goal is. I would *love* to have such a simple way of defining goals in RL. RL rewards are often sparse, have multiple objectives, and are highly multidimensional. To this day defining a reward function is largely an exhaustive manual process, even with techniques such as intrinsic rewards. In the future I still expect some manual intervention to define the high-level goals, but I expect to do this in a

less invasive way; I shouldn't need to shape the reward. This becomes even more important when rewards are delayed.

Combining different types of learning

Is there a way to combine the exploratory benefits of multi-agent RL, the generalization of hierarchical RL, and expert guidance? Could you combine evolutionary or genetic learning with these RL techniques?

Pretrained agents and improving curriculum learning

I once read, I can't remember where, that learning is only possible when the gap between knowing and not knowing is small. I believe this to be true in RL, too. I don't expect a child to be able to drive a car; similarly, you should not expect an untrained agent to either. And overfitting the training to that one task is not the solution, since driving a car is more than just learning how to incorporate complex stereo vision and move a wheel. The agent needs a breadth of knowledge that can be provided by agents pretrained on specified curricula. You could even give them a grade at the end of it if you like.

Increased focus on real-life applications and benchmarks

I think that academia understands that real-life applicability is desirable, but I would like to see more emphasis placed on this. I can see algorithms that are overfitting the Atari benchmarks and that is by design, to be able to claim state-of-the-art performance.

Ethical standards

Ethical or moral machine learning and RL crosses into both industry and academia. There seems to be a melting pot of desire for advancing the discussion, but this is proving difficult. Maybe because it always turns into a philosophical debate or maybe because everyone has a different set of morals or contexts that makes it difficult to agree on what ethical conclusions to make. It's a really sensitive subject that I don't feel qualified to comment on. But that doesn't mean that you shouldn't consider it; you should. In everything you do.

Regarding the future, I don't feel like there has been a flood of support for even attempting to define ethical standards, let alone solving them. The

General Data Protection Regulation (GDPR) is one example of a governmental attempt to define the ethical use of someone's data. And although I totally agree with the idea, in practice it is too vague and has too much legal jargon to be useful. For example, article 7, conditions for consent of the GDPR, states:

> *Consent shall be presented in a manner which is clearly distinguishable from the other matters, in an intelligible and easily accessible form, using clear and plain language.*

Fair enough. But in that same exact page, point 4 reads:

> *When assessing whether consent is freely given, utmost account shall be taken of whether, inter alia, the performance of a contract, including the provision of a service, is conditional on consent to the processing of personal data that is not necessary for the performance of that contract.*

Clear and plain language? And as evidence of the complete disregard for the letter of this law, the clarifying recital number 32, conditions for consent, states "pre-ticked boxes or inactivity should not therefore constitute consent." I would hazard a guess at suggesting that nearly all GDPR opt-in forms on a website provide an "accept all" box, which of course, checks all the boxes before you've had chance to read what they are for.

This pseudosafety comes at a cost, too. I'm constantly annoyed at having to find a cross to remove the pop-up (what does the close button do, by the way, accept all or reject all?). It's like being back in the '90s with GeoCities websites filled to the brim with pop-ups.

So what has the GDPR got to do with machine learning and RL ethical standards? It represents the first and largest government-led attempt to rein in a practice, related to data, which it deemed to be unethical. Has it worked? A bit, maybe. It has curtailed some of the exploits of some large companies or governments that were attempting to steer important conversations about democracy and freedom. But at what cost? Is there a better way? Probably, but I don't think there is a single right answer.

For more general applications of machine learning and RL, I feel like this will be a slow burner; it won't become important until it smacks people in

the face, a bit like climate change. I think there will be success at a grass-roots level, however, a bit like how ethical hacking is forcing companies to invest in digital security. All businesses want to be safe and secure, but it takes the challenge to actually do anything about it. The same will be true for data and the models derived from it. Individuals or groups of loosely affiliated individuals will be passionate enough to attempt to hold businesses or governments to account.

And then there are people like you. The practitioners. The engineers. The people building these solutions. You are first in line to observe the impact of your work, so there is a great responsibility placed upon you to decide whether your algorithm is ethical. This is the burden of knowledge; use it wisely and make good decisions.

Concluding Remarks

I find it ironic that I finish a book about optimizing sequential decision making with a remark asking you to make good decisions. Who knows, maybe in the future we can use RL to make the ethical decisions for us.

Until recently, RL has been primarily a subject of academic interest. The aim of this book was to try to bring some of that knowledge to industry and business, to continue the journey of improving productivity through automation. RL has the potential to optimize strategic-level decision processes using goals directly derived from business metrics. This means that RL could impact entire business structures, from factory floor workers to CEOs.

This poses unique technical and ethical challenges. Like how decisions can have long-lasting consequences that are not known at the time and how an automatic strategy can impact people. But this shouldn't prevent you from experimenting to see how RL can help you in your situation. It really is just a tool; how you use it is what counts.

Next Steps

You've probably got lots of questions. That's good. Collate those questions and send them to me (see About the Author for contact details). But in general I recommend that you don't stop here. Try to find a practical

problem to solve, ideally at work. If you can persuade your business to support your research, maybe in your 20% time or maybe as a fully fledged time-boxed proof of concept, this gives you the time and the impetus to deal with the real-life challenges that you will face. If you stick to toy examples, I'm afraid that it will be too easy for you. If you can't do this in work time, then find a problem that is interesting to you, maybe related to another hobby. You probably won't get any dedicated time to do this, because life gets in the way, but hopefully it is rewarding and gives you some practical experience.

Beyond practical experience, I recommend you keep reading. Obviously RL is hugely dependent on machine learning, so any books on data science or machine learning are useful. There are some other RL books out there, too, which are very good, and should provide another perspective. And if you want to go full stack, then improve your general software and engineering skills.

To help cement everything you have learned, you need to use it. Experience is one way I've already mentioned, but mentorship, teaching, and general presenting is another great way to learn. If you have to explain something to someone else, especially when the format is formal, like a presentation, it forces you to consolidate your ideas and commit your models and generalizations to memory. Don't fall into the trap of thinking that you are not experienced enough to talk about it. You are; you are an expert in the experiences you have had. It is very likely that some of your experiences can generalize to RL to help apply, explain, or teach these concepts in a way that nobody else could. Give it a go!

Now It's Your Turn

With that, I'll hand it over to you.

As you know, you can find more resources on the accompanying website. But I also want to hear from you. I want to hear about the projects that you are working on, your challenges, your tips and tricks, and your ideas for the future of RL. I especially want to talk to you if you are thinking about applying RL in industry or you are struggling with a problem. I can make your life easier with solutions, tips, and assessments to reduce effort, reduce risks, and improve performance. My full contact details are printed in About the Author.

Good luck!

Further Reading

- Debugging:
  - Agans, David J. 2002. *Debugging: The 9 Indispensable Rules for Finding Even the Most Elusive Software and Hardware Problems*. AMACOM.
  - Fowler, Martin. 2018. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional.
  - Feathers, Michael C. 2004. *Working Effectively with Legacy Code*. Prentice Hall PTR.
- Deep learning debugging:
  - Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- RL debugging:
  - Excellent Reddit thread, which led to a lot of this information.[11]
  - John Schulman's fantastic nuts and bolts presentation.[12]
  - Representative experience from Matthew Rahtz.[13]
- Research directions:
  - Many of the ideas were inspired by the paper by Dulac-Arnold et al.[14]
  - Offline RL.[15]
  - I personally loved Max Tegmark's book on the future for AI.[16]

[1] Engstrom, Logan, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. 2020. "Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO". ArXiv:2005.12729, May.

[2] Ibid.

[3] Cobbe, Karl et al. "Reinforcement Learning with Prediction-Based Rewards." OpenAI. 31 October 2018.

[4] "Lessons Learned Reproducing a Deep Reinforcement Learning Paper". n.d. Accessed 17 August 2020.

[5] Ng, Andrew. "Nuts and Bolts of Applying Deep Learning". 2016.

6 Guo, Xiaoxiao, Satinder Singh, Honglak Lee, Richard L Lewis, and Xiaoshi Wang. 2014. "Deep Learning for Real-Time Atari Game Play Using Offline Monte-Carlo Tree Search Planning". *Advances in Neural Information Processing Systems* 27, edited by Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, 3338–3346. Curran Associates, Inc.

7 "Hype Cycle for Data Science and Machine Learning, 2019". n.d. Gartner. Accessed 17 August 2020.

8 "Gartner Top 10 Trends in Data and Analytics for 2020". n.d. Accessed 17 August 2020.

9 "Gartner Says AI Augmentation Will Create $2.9 Trillion of Business Value in 2021". n.d. Gartner. Accessed 17 August 2020.

10 "Too Many AI Researchers Think Real-World Problems Are Not Relevant". n.d. *MIT Technology Review*. Accessed 20 August 2020.

11 "Deep Reinforcement Learning Practical Tips". R/Reinforcementlearning, n.d., Reddit. Accessed 17 August 2020.

12 "Deep RL Bootcamp Lecture 6: Nuts and Bolts of Deep RL Experimentation". 2017.

13 "Lessons Learned Reproducing a Deep Reinforcement Learning Paper". n.d. Accessed 17 August 2020.

14 Dulac-Arnold, Gabriel, Daniel Mankowitz, and Todd Hester. 2019. "Challenges of Real-World Reinforcement Learning". ArXiv:1904.12901, April.

15 Levine, Sergey, Aviral Kumar, George Tucker, and Justin Fu. 2020. "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems". ArXiv:2005.01643, May.

16 Tegmark, Max. 2017. *Life 3.0: Being Human in the Age of Artificial Intelligence*. Penguin UK.